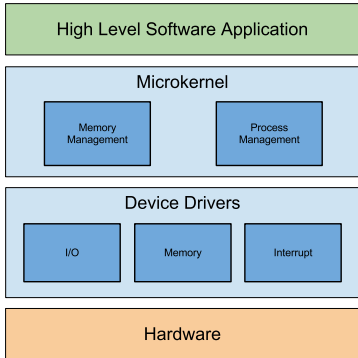


imal functionality usually limited to schedulers and memory management subunits. The remaining functionality including the middleware and device drivers are completely abstracted out of the microkernel. Microkernel based systems are infinitely more scalable and manageable since the required components can be added in dynamically. It's also much more secure as all the non-OS specific code has been moved to the client, and there is a precise demarcation



Microkernel OS

between the client and server memory. The drawback with this architecture is the overhead associated with the communication between the micro kernel module and the other modules moved out of the kernel. Context switching is also costly affair in this case as compared to the other architectural paradigms.

Examples of microkernel OSes are OS-9, C Executive, vxWorks, CMX-RTX, Nucleus Plus, and QNX.

Kernel

Like any other OS, the heart of an embedded OS is its kernel. The kernel is the nerve center of the operating system and is responsible for:

- 1 Booting
- 2 Task Scheduling
- 3 Memory Management
- 4 I/O System Management
- 5 Standard Function Libraries

Embedded OSes will differ in which components they include from the models shown above, but lowest common denominator is the kernel, all the OSes have a kernel at the very least. We'll discuss all the kernel components in the following sections. Before that a heads up about the standard function libraries, usually such systems have very limited space so there is no question of a full featured library likes of libc existing in the memory. The prerequisite of a function being included in the library is that they're small and really important.

In most of embedded systems, the bootloader will prep the kernel and load it into the memory. Kernel is responsible for initialising the system and ports, setting the global data items, initiating hardware timers and acti-